

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

Refer to the following GitHub Repository for the Code Notebook of this experiment:

<https://github.com/D-Rockie/Deep-Learning-Lab-2026.git>

7. Experimental Results

The dataset used in this experiment consisted of 1372 banknote samples described by 4 numerical features (variance, skewness, kurtosis, and entropy), with no missing values in any column. Of the 1372 samples, 762 belonged to the authentic class and 610 to the forged class, giving a reasonably balanced dataset that did not require any resampling. After standardizing all four features to zero mean and unit variance, the data was split into 1098 training samples and 274 testing samples using an 80/20 stratified split, so that both subsets preserved the original class ratio.

The perceptron was trained for up to 100 epochs with a learning rate of $\eta = 0.01$. Training began with 59 misclassified samples in the first epoch, which fell rapidly over the next 15–20 epochs before settling into an oscillation, fluctuating roughly between 9 and 20 misclassified samples for the remainder of training rather than reaching zero. This behaviour is consistent with the dataset not being perfectly linearly separable – a single decision boundary cannot fully separate the two classes, so the perceptron rule keeps adjusting the weights indefinitely rather than converging. After 100 epochs, the final weight vector and bias were:

$$\mathbf{w} = [-0.2702, -0.2997, -0.2595, -0.0391], \quad b = -0.12$$

Evaluating this trained model on the held-out test set gave the following results:

- Accuracy: 0.9818
- Precision: 0.9680
- Recall: 0.9918
- F1-score: 0.9798

The corresponding confusion matrix was:

$$\begin{bmatrix} 148 & 4 \\ 1 & 121 \end{bmatrix}$$

which corresponds to 148 true negatives, 4 false positives, 1 false negative, and 121 true positives out of 274 test samples. Despite the perceptron never fully converging during training, the test-time performance is strong, which suggests that the small region of class overlap identified earlier in the scatter plot affects only a minority of borderline samples, while the majority of the data remains cleanly separable by the learned hyperplane.

To study the effect of the learning rate, the same model was retrained with $\eta = 0.001, 0.01$, and 0.1 , keeping all other settings fixed. The three runs produced very similar outcomes: final training errors of 15, 14, and 16 misclassifications respectively, and test accuracies of 0.9818, 0.9818, and 0.9781. This similarity is expected given how the perceptron rule works – since the step activation function only depends on the sign of $\mathbf{w}^T \mathbf{x} + b$, scaling the learning rate mainly changes how large the weight updates are in magnitude, not the direction the weights move in, so all three runs converge toward comparable decision boundaries.

Finally, the from-scratch implementation was compared against Scikit-learn’s built-in `Perceptron` class trained under the same conditions. The two models produced nearly identical results:

Metric	Our Perceptron	Sklearn Perceptron
Accuracy	0.9818	0.9818
Precision	0.9680	1.0000
Recall	0.9918	0.9590
F1-score	0.9798	0.9791

The learned weight vectors were also close in both direction and magnitude

ours: $[-0.2702, -0.2997, -0.2595, -0.0391]$, bias -0.12 ;

sklearn's: $[-0.2445, -0.2735, -0.2758, -0.0046]$, bias -0.13 ,

which supports the correctness of the from-scratch implementation. The only meaningful difference is in the precision/recall trade-off – sklearn's version achieves perfect precision but slightly lower recall, meaning it is marginally more conservative about labelling a note as forged, whereas our implementation catches slightly more forged notes at the cost of a few more false positives.

8. Generated Plots with Interpretations

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

Plot 1: Feature Histograms

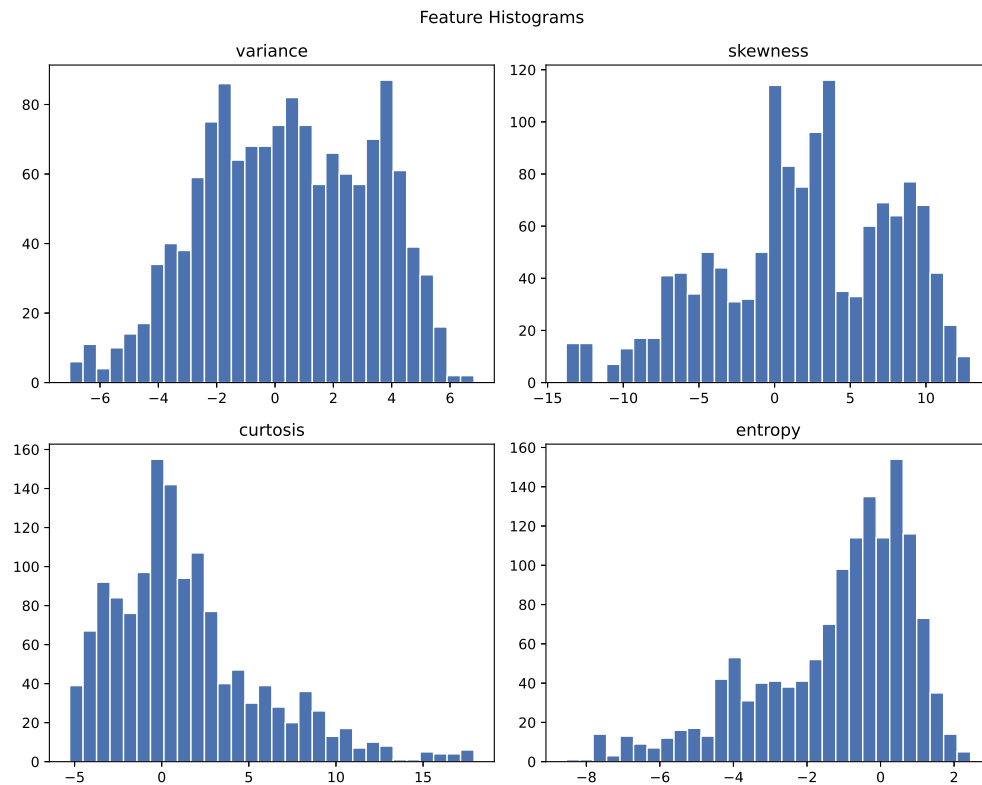


Figure 1: Plot 1: Histograms

Variance and skewness both show a multi-peaked shape rather than a single smooth bell curve, or a one sided skewed curve, which is a visual sign that two overlapping populations (genuine and forged notes) are mixed together in the same histogram. Kurtosis is heavily right-skewed with a long tail extending toward high values, while entropy is left-skewed and concentrated near its upper range. Therefore, these both patterns are says that these two features carry comparatively less class-separating information.

Plot 2: Correlation Heatmap

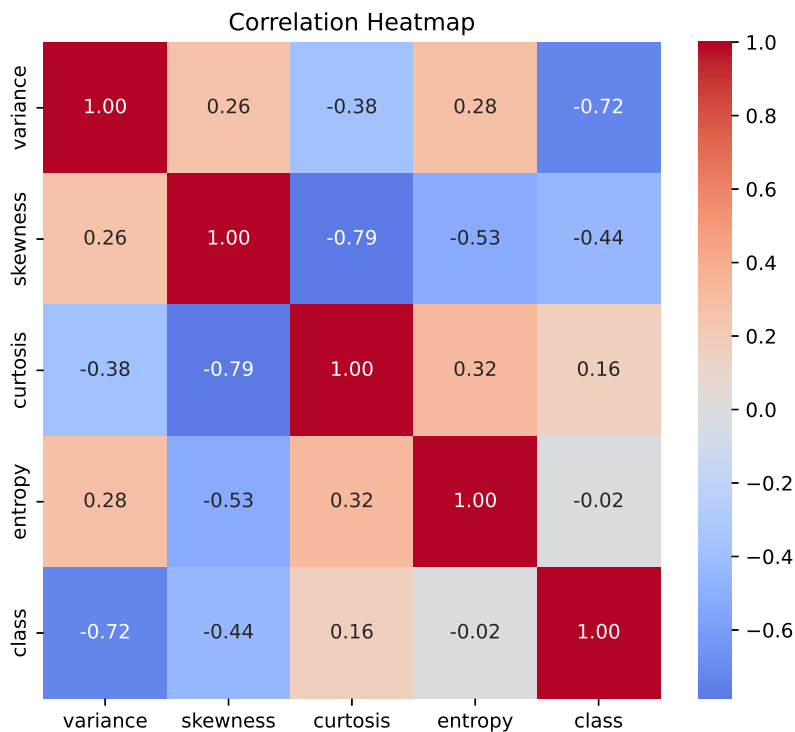


Figure 2: Pairwise Pearson correlation between the 4 features and the class label.

Variance has the strongest correlation with class (-0.72), followed by skewness (-0.44), meaning these two features individually carry the most information about whether a note is genuine or forged. Entropy correlates almost negligibly with class (-0.02), while curtosis shows a weak positive correlation (0.16) – notably in the opposite direction to variance and skewness. Skewness and curtosis are also strongly correlated with each other (-0.79), indicating some redundancy between these two features.

Plot 3: Scatter Plot (Variance vs Skewness)

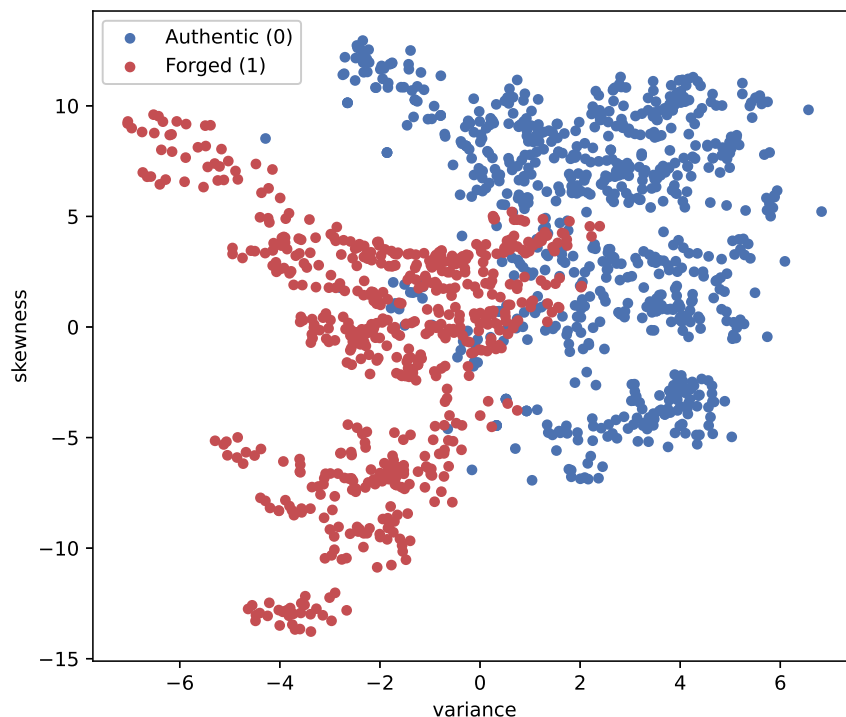


Figure 3: The two most class-correlated features, colored by class.

The two classes form visibly distinct, elongated clusters – forged notes concentrate toward lower variance and skewness, authentic notes toward higher values of both – but a clear overlap region exists around $\text{variance} \approx -2$ to $+2$, where the two classes intermix. This overlap indicates the classes are close to, but not perfectly, linearly separable using these two features, which directly explains why the perceptron’s training error (given by Plot 5) never reaches zero. The curved, non-circular shape of each cluster also shows why a single straight decision boundary can only approximate the true separation, but cannot separate the classes permanently.

Plot 4: Boxplots

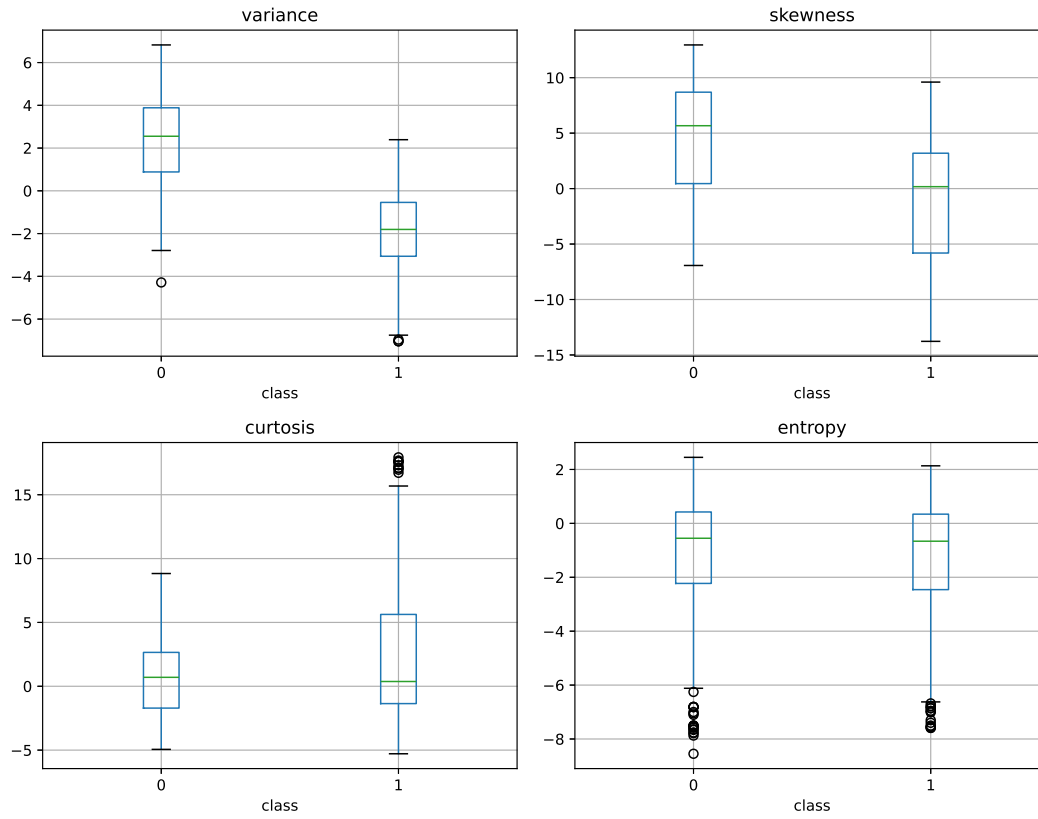


Figure 4: Feature spread and outliers, split by class.

Variance and skewness show the clearest separation in median and interquartile range between the two classes, consistent with their strong correlation with class shown earlier. Kurtosis and entropy boxplots overlap heavily between the two classes, confirming that these features are comparatively weak discriminators when used individually, even though they still contribute when combined with the others in the full 4-feature model.

Plot 5: Training Error vs Epoch

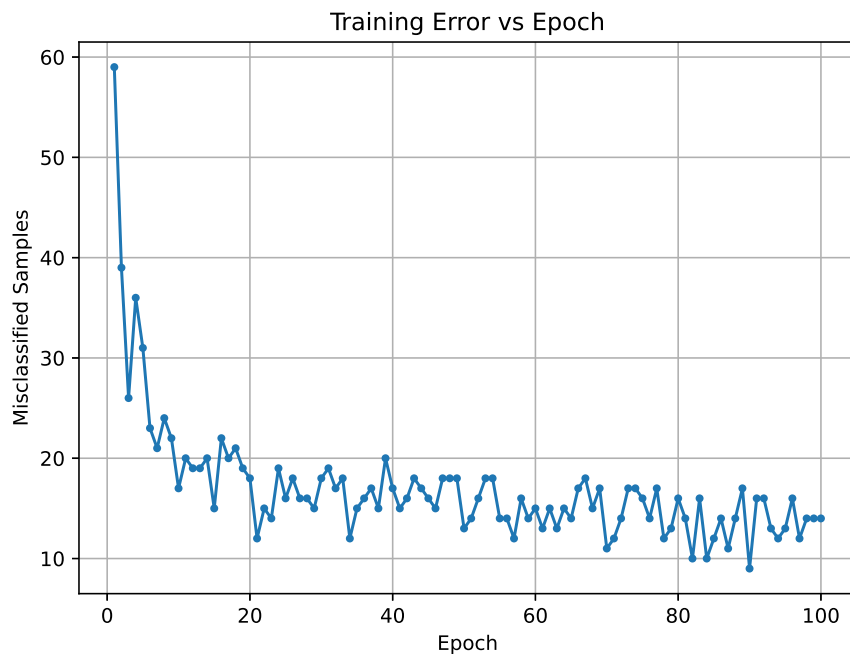


Figure 5: Number of misclassified training samples per epoch, $\eta = 0.01$.

The training error drops sharply within the first 10–15 epochs and then oscillates in a narrow band for the remaining epochs, and never reaches zero. This is expected behaviour for data that is not perfectly linearly separable: the perceptron convergence theorem only guarantees convergence to zero error when a separating hyperplane exists, which the overlap seen in Plot 3 shows is why the training never reaches zero.

Plot 6: Weight Evolution

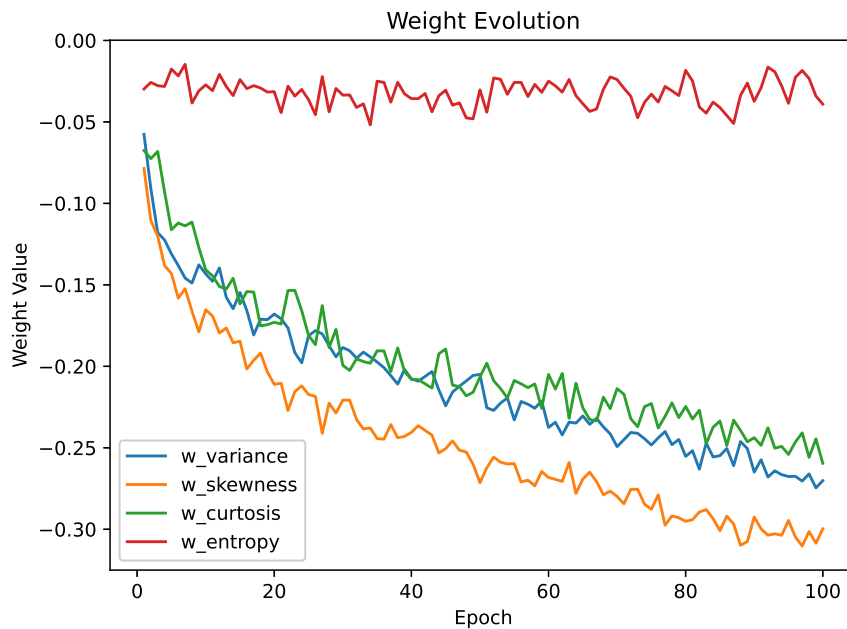


Figure 6: Value of each of the 4 weights across training epochs.

All four weights move quickly during the first few epochs and then plateau into small oscillations rather than settling at a single fixed point. This is a direct consequence of the training error itself oscillating instead of reaching zero. The weight corresponding to entropy stays closest to zero throughout training, consistent with its near-zero correlation with the class label as observed in the correlation heatmap.

Plot 7: Bias Evolution

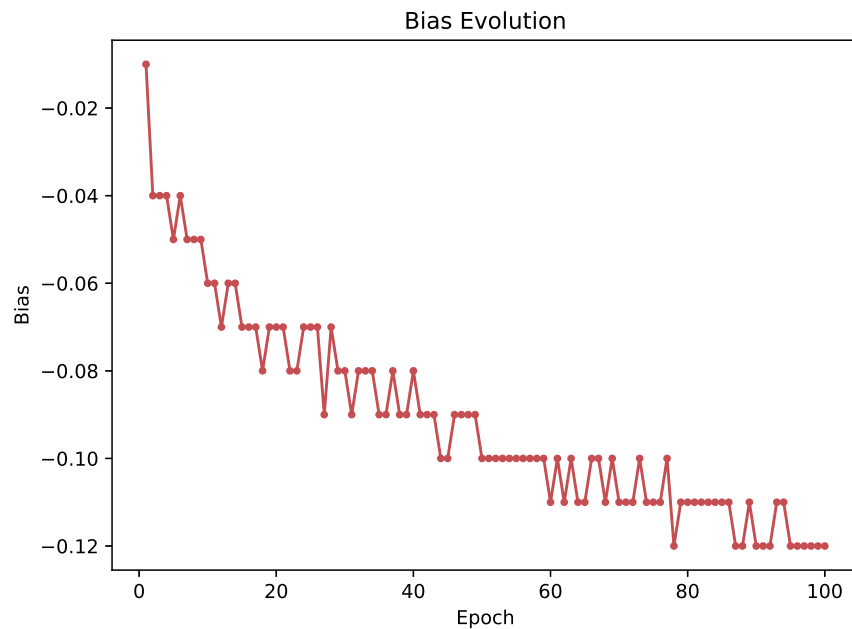


Figure 7: Bias term across training epochs.

The bias steadily shifts and then stabilizes, adjusting the decision boundary to account for the slightly unequal class sizes in the training split. Its evolution is smoother than the individual feature weights, since the bias term is updated by a fixed step ($\eta \times \text{error}$) rather than one scaled by an input feature value.

Plot 8: Confusion Matrix (Testing dataset)

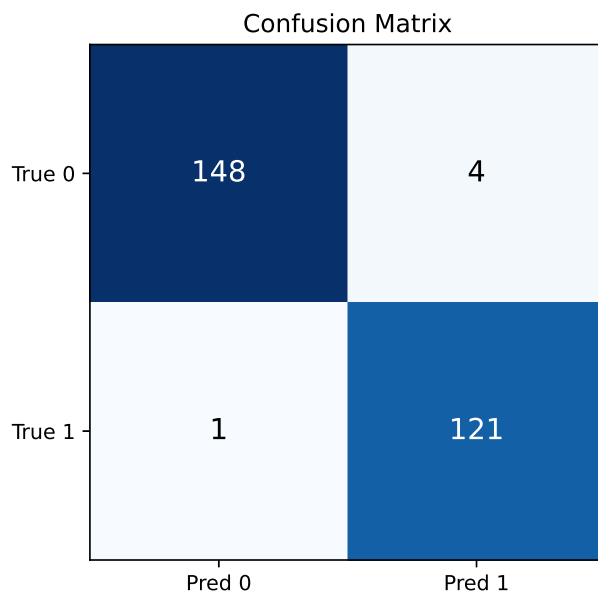


Figure 8: True vs predicted labels on the held-out test samples.

The model makes very few errors overall, with a noticeably asymmetric error pattern: more false positives (authentic notes flagged as forged) than false negatives (forged notes passed as authentic). In a real banknote-screening context this asymmetry is actually favourable, since a false negative – letting a forged note through – is typically the costlier mistake, and this model leans toward the safer error type.

Plot 9: Learning Rate Comparison

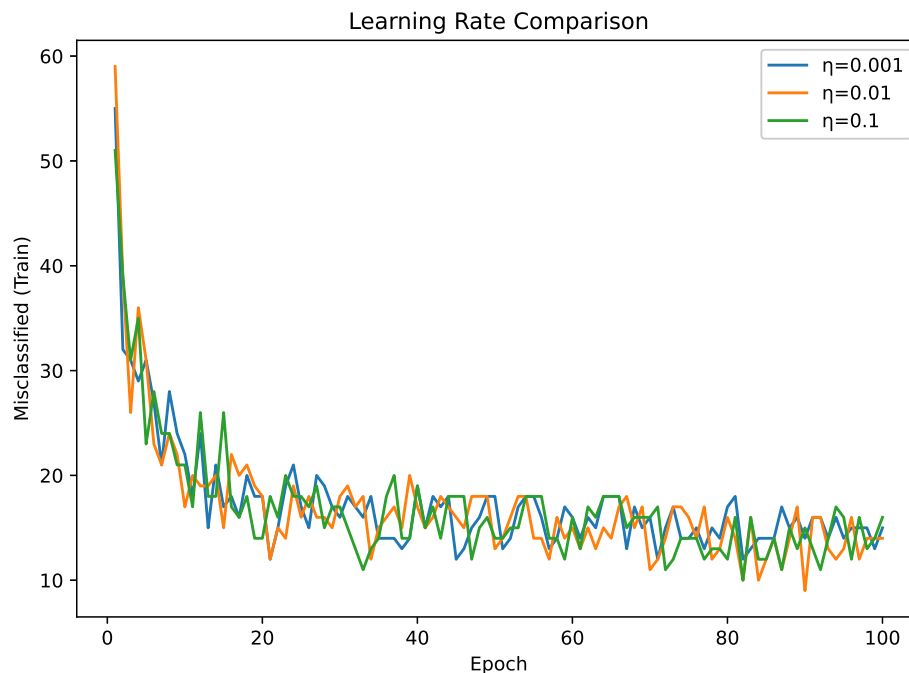


Figure 9: Training error vs epoch for $\eta = 0.001, 0.01, 0.1$.

All three learning rates converge to a similar residual error band, since none of them can perfectly separate this data, the ceiling on convergence is set by data separability, not by the step size. The visible difference between the curves is mainly in smoothness: the largest learning rate produces a noticeably jumpier curve since each weight update takes a bigger step, while the smallest learning rate is smoother but slower to leave the initial high-error region.

Plot 10: Decision Boundary (2-Feature Perceptron)

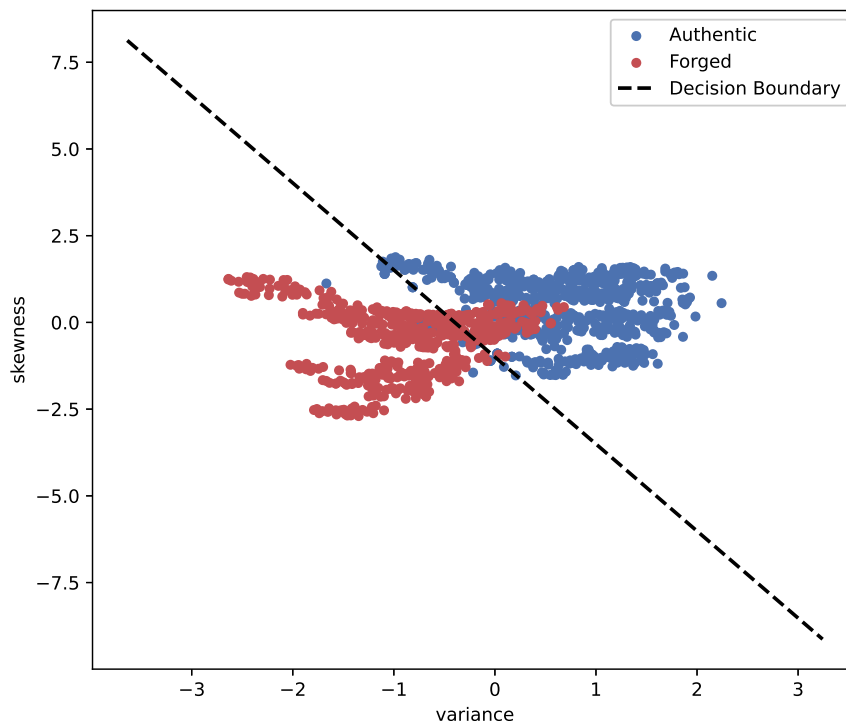


Figure 10: Boundary learned using only variance and skewness (standardized), for visualization.

The learned boundary sits roughly where the scatter plot in Plot 3 suggests it should, but a band of misclassified points near the overlap region is unavoidable with only a straight line available to separate the classes. This 2-feature-only model achieves noticeably lower test accuracy than the full 4-feature model, quantifying how much kurtosis and entropy still contribute to classification despite their individually weak correlation with the class label.

9. Performance Tables

Training Summary

Dataset Size	1372 (1098 train / 274 test)
Train/Test Split	80% / 20%, stratified
Learning Rate	0.01
Epochs	100 (did not reach zero error)
Final Weights	$[-0.2702, -0.2997, -0.2595, -0.0391]$
Final Bias	-0.12
Accuracy	0.9818
Precision	0.9680
Recall	0.9918
F1-score	0.9798

Epoch-wise Learning

Epoch	Errors	W_variance	W_skewness	W_curtosis	W_entropy	Bias
1	59	-0.0576	-0.0786	-0.0676	-0.0298	-0.01
5	31	-0.1310	-0.1430	-0.1162	-0.0175	-0.05
25	16	-0.1813	-0.2173	-0.1810	-0.0363	-0.07
50	13	-0.2050	-0.2714	-0.2065	-0.0303	-0.10
100	14	-0.2702	-0.2997	-0.2595	-0.0391	-0.12

10. Conclusion

In this experiment we implemented a Single Layer Perceptron from scratch using NumPy. The model was trained on the UCI Banknote Authentication dataset to distinguish genuine banknotes from forged ones based on four wavelet-transform-derived features: variance, skewness, kurtosis, and entropy.

Exploratory analysis showed that variance and skewness carry the strongest relationship with the class label, while entropy contributes almost no discriminative signal on its own. The scatter plot of the two strongest features also revealed a small region of overlap between the two classes, indicating the dataset is not perfectly linearly separable. This was confirmed directly during training: the misclassification count never reached zero, instead settling into a narrow oscillating band after the first 15–20 epochs, which is the expected behaviour of the perceptron learning rule on data lacking a perfect separating hyperplane.

Despite this, the trained model performed strongly on unseen data, achieving 98.18% accuracy, 96.80% precision, 99.18% recall, and an F1-score of 0.9798 on the held-out test set. Comparing learning rates of 0.001, 0.01, and 0.1 showed that final test accuracy was largely insensitive to this choice, since the step activation function depends only on the sign of the weighted sum – scaling the learning rate changes the magnitude of the weight updates but not their direction, so all three rates converged toward comparable decision boundaries. Validating the implementation against Scikit-learn’s built-in Perceptron produced matching accuracy and closely aligned weight vectors, confirming that the from-scratch model was implemented correctly.

Overall, this experiment demonstrated both the capabilities and the inherent limitations of

a single-layer, step-activation model: it can achieve strong accuracy even on data that is only approximately linearly separable, but it cannot fully converge on such data.

11. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.